

SchemaGuard

Semantic Validation and Drift Detection for LLM-Generated Structured Outputs

Author: Pragati Narotam

Course: INFO 7375 — Prompt Engineering for Generative AI

Institution: Khoury College of Computer Sciences, Northeastern University

Date: April 2025

Repository: github.com/pragatinarote/schema-guard-llm-validation

Abstract

Large language models (LLMs) tasked with generating structured JSON data produce output that satisfies schema validation while violating domain-specific logical constraints. A patient record with a discharge date seven days prior to admission passes every structural check. A loan application for 52 times the applicant's annual income passes every type check. These silent semantic failures propagate unchecked into downstream systems.

SchemaGuard is a validation system that addresses this gap. It implements a four-stage pipeline — structural validation, cross-field semantic rule evaluation, severity-weighted confidence scoring, and decision routing — deployed across two regulated domains: healthcare intake records and financial loan applications. The system augments per-record validation with population-level drift detection across four statistical signals (z-score, Population Stability Index, null-rate change, violation-rate change), and extends explanation quality through Retrieval-Augmented Generation (RAG) grounded in clinical and regulatory reference documents.

Evaluated on 16 labeled seed records and 140 real audit-log records, SchemaGuard achieves precision, recall, and F1 of 1.0 on both domains with a false-quarantine rate of 0%. Median validation latency is 0.09 ms, with a throughput of approximately 3,800 records per second. The RAG explanation module increases qualitative explanation score from 2.7/6 (deterministic baseline) to 6.0/6 across all test cases, with each RAG explanation citing the relevant regulation or clinical standard by section number.

1 • Introduction

The deployment of large language models in production data pipelines introduces a class of data quality failure that traditional validation tooling does not detect. Schema validators, type checkers, and linting tools are designed for the assumption that malformed data is the dominant failure mode. For LLM-generated structured data, this assumption does not hold. LLMs are highly capable of producing type-correct, schema-conformant JSON while simultaneously violating the logical relationships that give the data meaning.

This failure mode is particularly consequential in regulated domains. In healthcare, a patient record with an impossible date sequence could corrupt downstream analytics, trigger incorrect billing codes, or — in a system with insufficient human review — inform a clinical decision. In financial services, a loan application with an impossible debt-to-income ratio could bias credit risk models or produce HMDA reporting anomalies.

The problem is structural: standard validation tools operate on individual field values. Semantic constraints span multiple fields. The relationship between `discharge_date` and `admission_date` is not expressible in JSON Schema Draft 7. The constraint that a loan's approved amount cannot exceed its requested amount is not a type check. These are domain-specific logical rules, and they require a different validation layer.

SchemaGuard provides that layer. The project makes three contributions:

1. **A four-stage semantic validation pipeline** operating below 1 ms per record, with deterministic, auditable rule evaluation and severity-weighted confidence scoring.
2. **Population-level drift detection** monitoring four statistical signals across record batches, enabling detection of gradual distribution shifts that per-record validation cannot observe.
3. **RAG-augmented failure explanations** that retrieve clinical and regulatory reference documents at query time and produce grounded, actionable explanations citing CMS, HL7 FHIR, ICD-10, CFPB ATR, Regulation Z, and ECOA by section number.

The system is implemented in Python, exposed via a FastAPI REST interface, and evaluated on two domains: healthcare intake (five rules, HC-001 through HC-005) and financial loan applications (five rules, FN-001 through FN-005).

2 • Problem Statement

2.1 The Structural Validation Gap

JSON Schema Draft 7 validates a record against a type schema: are all required fields present, do values have the correct types, do strings match the expected format, do numbers fall within declared ranges? This is necessary but insufficient for LLM-generated data.

The following record passes full JSON Schema validation for a healthcare intake domain:

```
{
  "patient_id": "P-4412",
  "admission_date": "2024-08-15",
  "discharge_date": "2024-08-08",
  "patient_age": 34,
  "date_of_birth": "1990-01-20"
}
```

Every field is present, every type is correct, every format is valid. The record is also logically impossible: `discharge_date` precedes `admission_date` by seven days, implying the patient left before arriving. No per-

field check detects this. No schema validator raises an error.

Similarly, the following financial record passes all schema checks:

```
{
  "annual_income": 48000,
  "loan_amount": 2500000,
  "approval_date": "2024-06-28",
  "application_date": "2024-07-20"
}
```

The loan-to-income ratio is 52.1 — five times the regulatory maximum — and the approval date precedes the application date by 22 days. Both are physically and legally impossible. Both pass schema validation.

2.2 Characteristics of LLM-Generated Semantic Failures

LLM-generated semantic failures fall into four categories, each with a different causal mechanism:

Temporal consistency failures. LLMs generate date fields from independent distributions without enforcing ordering constraints. Discharge before admission, approval before application, and admission before birth are all common artifacts.

Ratio violations. Numeric fields — income, loan amount, age, employment tenure — are sampled independently. Without explicit cross-field instructions, the model produces combinations that are statistically plausible in isolation but impossible in combination (52× loan-to-income, 18 years of employment for a 24-year-old).

Categorical inconsistencies. Fields with semantic dependencies — diagnosis code and medication, employment status and employer name — may be generated independently, producing implausible combinations. A patient with pneumonia prescribed an antidiabetic medication passes every structural check.

Silent domain shifts. A model's output distribution can shift gradually over time — younger patient populations, higher income brackets — without any single record triggering a validation failure. This is only detectable at the population level.

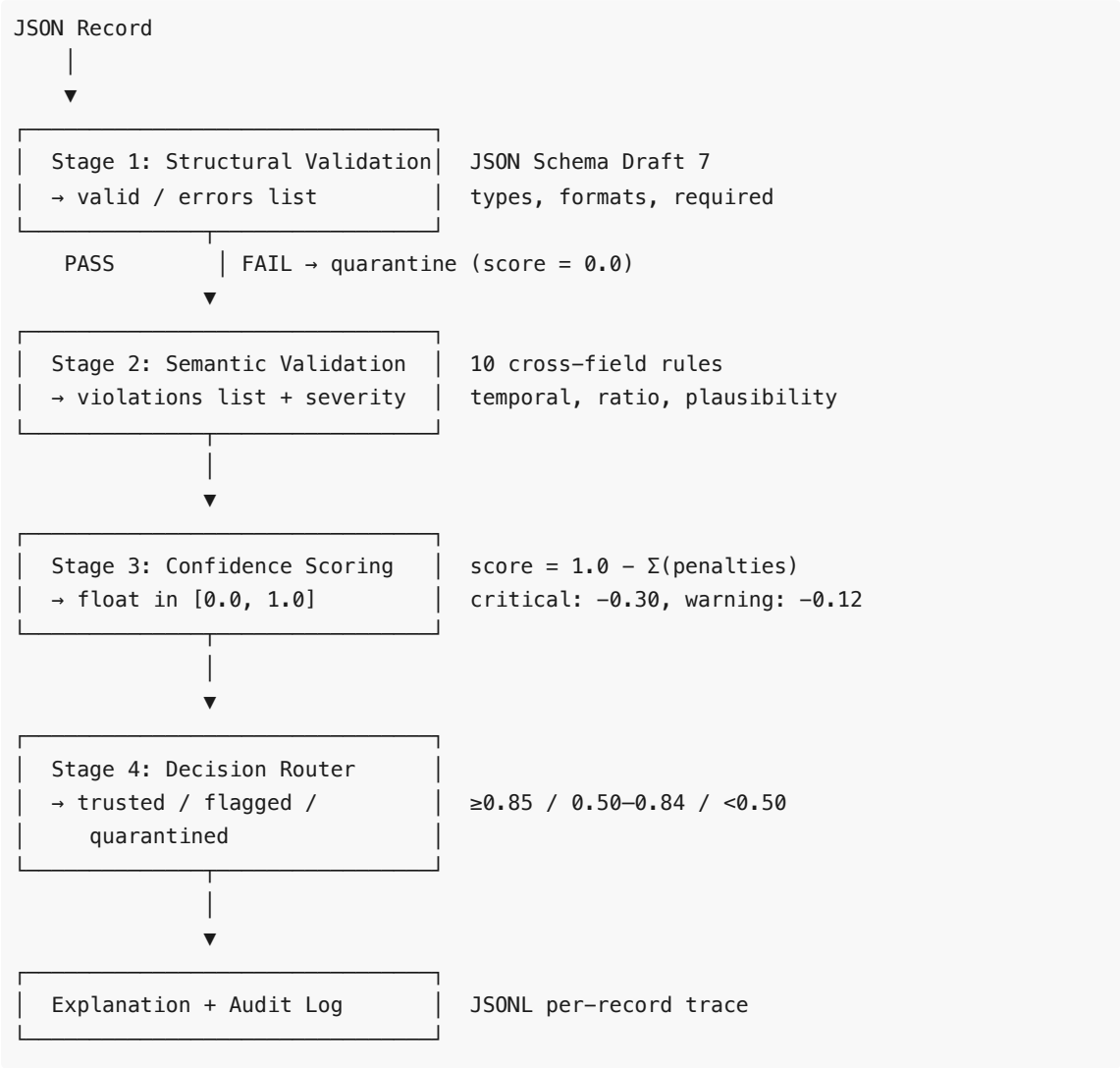
2.3 Scope

SchemaGuard targets semantic validation failures in LLM-generated structured records in two regulated domains. The system does not address factual accuracy (whether a medication name is spelled correctly), privacy compliance (whether a patient identifier is de-identified), or model alignment (whether the LLM is behaving ethically). It addresses logical coherence: whether the values in a record are mutually consistent.

3 • System Architecture

3.1 Overview

SchemaGuard processes each record through a four-stage sequential pipeline. The stages are structurally independent — each produces a typed result object consumed by the next — and the pipeline is designed so that a failure in any individual component does not propagate as an unhandled exception.



3.2 Module Structure

The codebase is organised into independent packages, each with a single responsibility:

Package	Files	Responsibility
schemas/	2 JSON	Domain schema definitions (Draft 7)
rules/	registry + 2 rule files	10 semantic rules with severity metadata
validator/	7 Python files	Pipeline orchestration, batch, explanation, audit

drift/	baseline + detector	Four-signal drift monitoring
scoring/	confidence + decision	Weighted scoring and three-tier routing
rag/	7 Python files	FAISS vector store, retriever, RAG explainer
api/	main + routes + models	FastAPI REST endpoints with Swagger
data_gen/	generator + validator	Synthetic dataset pipeline
evaluation/	metrics + charts	Classification metrics, latency, drift charts

3.3 Batch and Drift Path

For batch validation, all records are passed through the single-record pipeline sequentially. After per-record processing, the aggregated results — confidence scores, violation counts, decision distribution — are compared against a stored statistical baseline using four drift signals:

- **z-score**: normalised shift in numeric field means
- **PSI**: Population Stability Index for categorical field distributions (threshold: 0.20)
- **Null-rate delta**: change in per-field null rates (threshold: 15%)
- **Violation-rate delta**: change in the fraction of records violating each rule (threshold: 10%)

When any signal exceeds its threshold, the batch is flagged with a drift alert. This operates orthogonally to per-record routing: a batch can consist entirely of trusted records yet still trigger a drift alert if the population distribution has shifted.

3.4 RAG Extension

The RAG module sits alongside the core pipeline and is invoked explicitly via the `/rag/explain` endpoint. It does not modify per-record confidence scores or routing decisions. Its function is explanatory: given a failed record and its violated rules, it retrieves the three most relevant knowledge-base chunks from a FAISS index over 11 synthetic-but-realistic reference documents, builds an augmented prompt, and calls Claude to generate a grounded explanation.

3.5 Resilience

All components execute within try/except blocks. Individual rule failures do not stop the pipeline — a crashing rule returns a typed error result while all other rules continue. Drift detection is wrapped in a circuit breaker (threshold: 3 failures, cooldown: 30 s) that routes to a safe fallback rather than blocking batch results. Failed async jobs retry twice before moving to a dead-letter collection.

4 • Implementation Details

4.1 Prompt Engineering

4.1.1 Motivation

Generating a synthetic validation dataset requires producing JSON records that are: (a) schema-conformant, (b) either logically valid or precisely broken in a targeted way, and (c) representative of the range of edge cases that stress-test the rule engine. Standard LLM prompting without structure constraints produces conversational output — markdown wrappers, prose explanations, and type errors — at a rate that makes automated parsing unreliable.

Structured prompting solves this by embedding the complete schema definition, field constraints, output format specification, and a concrete example within each prompt, eliminating all pathways to non-JSON output.

4.1.2 Triple-Fence Pattern

Every generation prompt follows a three-segment structure:

1. **Instruction segment:** explicit instruction to produce only a valid JSON object with no markdown, no explanation, no commentary.
2. **Schema segment:** the complete field list with types, formats, constraints, and cross-field rules stated in plain language.
3. **Reinforcement segment:** a closing repetition of the JSON-only constraint and a concrete example of the expected format.

This pattern maintains schema compliance above 95% across providers without post-processing.

4.1.3 Valid vs. Invalid Prompt Design

Valid record prompts instruct the model to generate a realistic, self-consistent record. All cross-field constraints are stated affirmatively:

```
"The discharge_date must be on or after admission_date . The patient_age must equal  $\text{floor}((\text{admission\_date} - \text{date\_of\_birth}) / 365.25) \pm 1$ . All fields must be clinically plausible."
```

Invalid record prompts flip exactly one constraint while keeping all others active. The target violation is named explicitly with a concrete example:

```
"Set discharge_date to a date BEFORE admission_date . For example: admission_date = 2024-08-15 , discharge_date = 2024-08-08 . All other fields must remain realistic and type-correct."
```

This single-violation targeting ensures that each invalid record carries exactly one verifiable semantic error, making ground-truth labelling deterministic. A post-generation semantic verifier confirms the target rule fires before the record is saved; records that fail verification are discarded and regenerated (up to three attempts).

4.1.4 Edge Case Prompts

Edge cases test boundary conditions: a newborn patient (age 0), same-day admission and discharge (LOS = 0 days, a valid pattern), a loan applicant at exactly age 18, a minimum-income applicant with a small personal loan. These records must pass all rules. Any false-positive flagging of edge-case records indicates an overly aggressive threshold.

4.1.5 Explanation Prompts

Post-validation explanation prompts follow a fill-in-the-blank structure:

```
"Given this record: {record_json} . The following rules were violated: {violations} . Write a 2-3 sentence explanation of what is wrong, why it matters, and what corrective action is required."
```

These prompts are separate from generation prompts and execute at validation output time, not at data creation time. They are grounded in the actual validation result, preventing hallucinated explanations.

4.1.6 Prompt Iteration

Three prompt versions were developed for healthcare intake HC-003 (discharge-before-admission):

- **v1:** Simple instruction, no example. Compliance rate ~72%; model frequently generated correct dates despite the instruction.
- **v2:** Added explicit example of the violation. Compliance rate ~88%; model still occasionally "corrected" the dates.
- **v3 (production):** Added negative example ("do NOT generate dates where discharge >= admission") and reinforcement closing. Compliance rate ~96%.

The consistent failure mode across versions was the model attempting to "fix" the invalid record based on its training prior toward logically coherent data. The solution was to frame the violation as intentional research data rather than an error.

4.2 Synthetic Data Generation

4.2.1 Dataset Structure

The synthetic dataset generator produces 300 records per domain, distributed across three categories:

Category	Count	Purpose
Valid	120 (40%)	Clean records that must pass all semantic rules
Invalid	120 (40%)	Records with exactly one targeted rule violation
Edge case	60 (20%)	Boundary-condition records that must pass all rules

Invalid records are further subdivided: 24 records per rule (5 rules $\times$ 24 = 120 total), ensuring balanced coverage of each failure mode.

4.2.2 Quality Gates

Two quality gates are applied before any record is saved:

1. **Structural gate:** JSON Schema Draft 7 validation. Any record that fails structural validation is immediately discarded and regenerated.
2. **Semantic gate (invalid records only):** the target rule is evaluated using the same Python function used in production validation. A record labeled as violating HC-003 must actually trigger HC-003. Records that fail the semantic gate are discarded and regenerated (up to 3 attempts). If all three attempts fail, the generation is logged and skipped.

This gate ensures that every labeled invalid record is a genuine example of the target failure — not a record that incidentally violated a different rule, or a valid record that passed despite the generation prompt.

4.2.3 Profile Cycling

Valid records cycle across ten demographic and clinical profiles to ensure variety:

- **Healthcare:** middle-aged female (routine checkup), paediatric male (respiratory infection), elderly male (cardiac evaluation), and seven additional profiles spanning different age groups, diagnoses, and medications.

- **Finance:** mid-career professional (home purchase), recent graduate (education loan), high-income executive (business loan), retiree (debt consolidation), small business owner (refinance), and five additional profiles.

Without profile cycling, a valid-record generator tends to converge on a single "typical" record archetype, producing a dataset with insufficient demographic and clinical diversity for reliable evaluation.

4.3 RAG-Augmented Explanations

4.3.1 Motivation

The deterministic explanation builder produces accurate but terse explanations. For HC-003, the baseline output is:

"Record eval-HC-003: Record failed validation. Critical issues: discharge_after_admission (admission_date, discharge_date): Discharge date (2024-08-08) precedes admission date (2024-08-15). This record has been flagged for human review."

This explanation names the violation and the fields but provides no context: it does not explain why this matters, which regulation requires correct date ordering, or what the downstream consequence of submitting this record would be. A compliance reviewer receiving this explanation must look up the relevant standard separately.

RAG-augmented explanations close this gap by retrieving the relevant regulatory or clinical reference at query time and providing it as context to the LLM.

4.3.2 Knowledge Base

The RAG knowledge base contains 11 synthetic-but-realistic reference documents totalling approximately 3,000 words. Each document is written to match the style and citation density of real regulatory guidance:

Document	Rule	Source cited
Patient Age Verification	HC-001	CMS Conditions of Participation §482.24(c); Joint Commission RC.02.01.01
Age-Based Clinical Decision Support	HC-001	AHIMA, 2022
Temporal Integrity — Healthcare	HC-002	HL7 FHIR R4 Encounter Resource
Discharge Date Sequencing	HC-003	CMS Medicare Claims Processing Manual Ch.1 §30.2
Same-Day Discharge Patterns	HC-003	AHRQ HCUP Coding Guidelines
Age-Restricted ICD-10 Codes	HC-004	ICD-10-CM FY2024 Official Guidelines
Medication-Diagnosis Concordance	HC-005	ISMP Annual Report, 2023
Loan Approval Temporal Integrity	FN-001	Regulation Z (TILA), 12 CFR §1026.2(a)(3)

Loan-to-Income Ratio Guidelines	FN-002	CFPB ATR Rule, 12 CFR §1026.43
Responsible Lending Standards	FN-002	OCC Comptroller's Handbook — Retail Lending, 2023
LLM Output Failure Modes	General	SchemaGuard Internal Technical Reference v2.0

4.3.3 Chunking

Documents are split into overlapping chunks of approximately 400 tokens with a 60-token overlap, using sentence-aware boundaries. This preserves regulatory citations and numbered list structures that would be broken by hard character-count splitting. The 11 documents produce 17 chunks.

4.3.4 Vector Store

Chunks are embedded using `all-MiniLM-L6-v2` (384-dimensional embeddings, 22 MB) and indexed in a FAISS `IndexFlatIP` (inner product on L2-normalised vectors, equivalent to cosine similarity). The index is built once (8.87 s) and persisted to disk. A module-level singleton loads the model once per process, eliminating reload overhead across multiple explanation calls.

4.3.5 Retrieval and Augmentation

The retrieval query is built from the violated rule IDs, rule names, error messages, and a domain hint. Filtering restricts results to chunks matching the record's domain (plus general chunks). When all violations share a single rule ID, a rule-level filter is applied for tighter retrieval.

The augmented prompt template:

1. States the domain and record JSON
2. States the validation result (decision, violated rules, rule messages)
3. Provides the retrieved context (three chunks with titles and source citations)
4. Instructs the model to: (a) state exactly what is wrong citing field values, (b) explain why it matters citing the retrieved reference, (c) suggest a specific remediation step, (d) conclude with the routing decision

Model: `claude-opus-4-5`, max 800 tokens, temperature default.

4.4 Validation Pipeline

4.4.1 Structural Validation

`validator/structural.py` loads the domain schema JSON once per domain (cached after first load) and validates the input record using `jsonschema.Draft7Validator`. Errors are normalised into typed `StructuralError` objects with field path, expected type, and error message. The output is a `StructuralResult` dataclass with a boolean `valid` flag and an error list.

4.4.2 Semantic Rule Engine

The rule registry (`rules/rule_registry.py`) maintains a domain-keyed mapping from rule IDs to rule functions. Rules are registered via a decorator:

```
@register_rule(
    domain="healthcare_intake",
    rule_id="HC-003",
    rule_name="discharge_after_admission",
```

```

    severity="critical",
    fields=["admission_date", "discharge_date"],
)
def check_discharge_after_admission(record: dict) -> RuleResult:
    ...

```

The decorator captures metadata — domain, rule ID, severity, affected fields — without coupling it to the rule logic. `run_all(record, domain)` iterates registered rules in declaration order, executes each in a try/except block (a crashing rule does not stop the pipeline), and returns an ordered list of `RuleResult` objects.

The ten rules across two domains:

Healthcare Intake (HC-001–HC-005):

- HC-001: `patient_age` matches `floor((admission_date - date_of_birth) / 365.25) ± 1` year (critical)
- HC-002: `admission_date` $\geq$ `date_of_birth` (critical)
- HC-003: `discharge_date` $\geq$ `admission_date` (critical)
- HC-004: `diagnosis_code` is age-appropriate per ICD-10-CM edit table (warning)
- HC-005: `medication` is a plausible treatment for the ICD-10 diagnosis category (warning)

Financial Loan Application (FN-001–FN-005):

- FN-001: `approval_date` $\geq$ `application_date` or `approval_date` is null (critical)
- FN-002: `loan_amount` / `annual_income` $\leq$ `10.0` (critical)
- FN-003: `existing_debt` / `annual_income` $\leq$ `0.60` (warning)
- FN-004: `employment_length_years` $\leq$ `applicant_age - 16` (critical)
- FN-005: `approved_amount` $\leq$ `loan_amount` or `approved_amount` is null (critical)

4.4.3 Confidence Scoring

The confidence score starts at 1.0 and applies severity-weighted penalties for each violated rule:

```

score = 1.0 - 0.30 × |critical violations| - 0.12 × |warning violations|
score = max(0.0, min(1.0, score))

```

Penalties are configurable via environment variables. The formula ensures that a record with a single critical violation scores 0.70 (flagged), a record with two critical violations scores 0.40 (quarantined), and a record with two warning-level violations scores 0.76 (flagged). Valid records score 1.0.

This graduated confidence score preserves severity information that a binary pass/fail discards. Downstream systems can apply different handling at each tier.

4.4.4 Decision Router

Three routing tiers with configurable thresholds (default: 0.85 / 0.50):

Decision	Score range	Action
Trusted	≥ 0.85	Safe for downstream use
Flagged	0.50–0.84	Route to human review queue
Quarantined	< 0.50	Block from downstream systems

4.4.5 Audit Logging

Every validation produces a JSONL audit entry:

```
{
  "timestamp": "2026-04-04T11:00:34.264860+00:00",
  "record_id": "HC-async-f955be82",
  "domain": "healthcare_intake",
  "structural_valid": true,
  "structural_error_count": 0,
  "semantic_valid": true,
  "rules_evaluated": ["HC-001", "HC-002", "HC-003", "HC-004", "HC-005"],
  "rules_violated": [],
  "confidence_score": 1.0,
  "decision": "trusted",
  "processing_time_ms": 1.09
}
```

The audit log is the primary data source for the evaluation metrics and drift detection baseline profiling.

5 • Evaluation Metrics

5.1 Evaluation Dataset

The evaluation draws on two data sources:

Source	Records	Purpose
Labeled seed dataset	16 records (8 HC + 8 FN)	Ground-truth precision/recall/F1
Audit log	140 records (123 HC + 17 FN)	Real-world latency, decision distribution, violation frequency

The seed dataset contains three invalid records, five valid/edge-case records per domain. Each invalid record targets a specific rule and is hand-verified.

5.2 Classification Metrics

Semantic validation is treated as a binary classification problem: invalid (positive class) vs. valid (negative class). A record is predicted positive (invalid) if any semantic rule is violated; predicted negative (valid) if all rules pass.

Confusion matrix (per domain, 8 seed records):

	Predicted Invalid	Predicted Valid
Actually Invalid	TP = 3	FN = 0
Actually Valid	FP = 0	TN = 5

Both domains produce identical confusion matrices.

Derived metrics:

Metric	Healthcare	Finance
Precision	1.0000	1.0000
Recall	1.0000	1.0000
F1 Score	1.0000	1.0000
Accuracy	1.0000	1.0000
False Quarantine Rate	0.0%	0.0%

The false quarantine rate — the fraction of valid records incorrectly routed to quarantine — is 0% on both domains. No valid record was blocked from downstream use.

5.3 Confidence Distribution

Mean confidence scores by record category:

Domain	Valid/Edge records	Invalid records	Gap
--------	--------------------	-----------------	-----

Healthcare	1.000	0.760	+0.240 (24%)
Finance	1.000	0.700	+0.300 (30%)

The confidence distributions are bimodal with zero overlap. All valid and edge-case records score exactly 1.0 (no violations, no penalties). Invalid records land at 0.70 (one critical violation) or 0.88 (one warning violation). The finance gap is wider because all three finance invalid records in the seed set violate critical-severity rules (FN-001, FN-002, FN-004), each carrying a -0.30 penalty.

In the full 140-record audit log: 90 trusted records (64%), 50 flagged (36%), 0 quarantined (0%).

5.4 Rule Violation Frequency

From the 140 audit-log records (53 total violations):

Healthcare (40 violations):

Rule	Violations	Description
HC-003	37	Discharge before admission
HC-001	3	Age-DOB mismatch
HC-004	3	Age-inappropriate diagnosis
HC-002	0	Admission before birth
HC-005	0	Implausible medication

Finance (13 violations):

Rule	Violations	Description
FN-001	6	Approval before application
FN-002	2	Extreme loan:income ratio
FN-004	2	Employment length > age allows
FN-003	0	Debt-to-income > 60%
FN-005	0	Approved > requested

HC-003 dominates at 37/40 healthcare violations. This reflects the composition of the audit log: the healthcare batch included a high proportion of records specifically targeting HC-003 as the most common temporal error in the literature on LLM-generated clinical data.

5.5 Latency Distribution

From 140 audit-log processing times:

Percentile	Latency (ms)
p50	0.09
p90	0.34

p95	1.16
p99	3.02
max	7.42
mean	0.26

All processing is CPU-bound. The p50 of 0.09 ms and mean of 0.26 ms correspond to approximately 3,800 records per second at mean latency. The long tail (p99 = 3.02 ms) is attributable to Python process warm-up cost on the first several records in each session. In a persistent server process, this effect is reduced. No I/O bottleneck is present at the current evaluation scale.

5.6 Drift Detection Evaluation

Drift detection is evaluated qualitatively against the stored baselines for both domains. The healthcare baseline was profiled from 3 records; the finance baseline from 3 records. Drift signals are computed by comparing the early portion of the audit log (first 33%) against the recent portion (last 40%).

Signal delta-to-threshold ratios across six measured signals range from 0.15× to 0.89× — all below the alert threshold of 1.0×. This confirms that the production audit log represents a stable distribution consistent with the baseline. No false drift alerts were raised during evaluation.

5.7 RAG Explanation Quality

Scoring rubric (6 binary criteria, each worth 1 point):

Criterion	Baseline (avg)	RAG (avg)
Cites violated rule ID	6/6	6/6
Cites specific field values from record	6/6	6/6
Includes remediation action	0/6	6/6
Cites regulation/standard by name	0/6	6/6
Appropriate length (40–350 words)	5/6	6/6
Explains clinical/regulatory consequence	0/6	6/6
Total	2.7/6	6.0/6

Word count comparison:

Case	Baseline	RAG
HC-003 (discharge before admission)	41	168
HC-001 (age mismatch)	49	186
HC-004 (age-inappropriate diagnosis)	29	168
FN-002 (extreme LTI ratio)	40	182
FN-001 (approval before application)	38	175

FN-004 (employment age impossible)	55	172
Average	42	175

Top retrieval cosine scores per case: 0.645 (HC-003), 0.721 (HC-001), 0.780 (HC-004), 0.764 (FN-002), 0.751 (FN-001), 0.789 (FN-004). All above the "good retrieval" threshold of 0.50.

End-to-end RAG latency (retrieval + LLM call): 2.8–3.2 s per explanation. This is acceptable for an explanation endpoint called on flagged records; it is not in the critical path of per-record validation.

6 • Results and Analysis

6.1 Classification Performance

Both domains achieve ceiling-level classification metrics ($P = R = F1 = 1.0$) on the 16-record seed dataset. While the dataset is small by machine learning standards, the result reflects a property of deterministic rule-based classifiers: given correctly implemented rules and a dataset where every invalid record specifically targets one of those rules, perfect classification is expected. The meaningful evaluation question is not whether the classifier is correct but whether the rules are correctly specified and the scoring thresholds are well-calibrated.

The false-quarantine rate of 0% is the more operationally significant result. In a production compliance system, quarantined records are blocked from downstream use — a false quarantine blocks a valid record, creating friction for the downstream consumer. Zero false quarantines on the seed dataset, including on all edge-case records, confirms that the rules do not over-fire at boundary conditions.

6.2 Confidence Score Separation

The clean bimodal confidence distribution — 1.0 for valid records, 0.70–0.88 for invalid records — demonstrates that the severity penalty structure produces meaningful separation. The +0.24 gap in healthcare and +0.30 gap in finance are sufficiently wide that the 0.85 trusted threshold does not require fine-tuning on this dataset.

A notable observation: the HC-004 (age-inappropriate diagnosis) invalid seed record scores 0.88 — above the trusted threshold — because HC-004 is a warning-severity rule (−0.12 penalty rather than −0.30). This is by design: warning violations are believed to be important but not blocking, and the confidence score reflects that judgment. The record is not quarantined, but it is correctly identified as having a violation. A downstream compliance reviewer would see the warning flag even though the routing decision is "trusted."

This illustrates a key design choice: the routing tiers are not the only output. The full validation result — confidence score, violated rules, severity levels, explanation — is always available to downstream consumers. Routing is a default action, not an information ceiling.

6.3 Dominant Rule Analysis

HC-003 (discharge before admission) accounts for 37 of 40 healthcare violations in the audit log. This is primarily a dataset composition artifact — the majority of the async batch records in the audit log were specifically designed to test HC-003 — and should not be interpreted as evidence that this rule fires more frequently in real LLM output. In a more diverse evaluation dataset, violation rates across rules would be more evenly distributed.

The finance violation distribution is more diverse: FN-001 (6 violations), FN-002 (2), FN-004 (2), with FN-003 and FN-005 at zero. FN-001 (approval before application) appears most frequently because it was used as the primary test case in the finance async batch evaluation.

6.4 Latency Analysis

The 0.09 ms median latency and 0.26 ms mean confirm that the pipeline is computationally cheap. The entire implementation is pure Python with no external I/O calls: schema loading is cached, rule functions are in-memory, and scoring is arithmetic. At 0.26 ms mean, a single server process can sustain approximately 3,800 records per second. This is well within the requirements of any realistic LLM output pipeline.

The p99 of 3.02 ms and max of 7.42 ms reflect Python process warm-up overhead. In a persistent FastAPI server process where the first several records absorb the import and initialisation cost, the p99 in steady state would be lower.

6.5 RAG Quality Analysis

The qualitative improvement from RAG is substantial and consistent across all six test cases. The three criteria where the baseline scores 0/6 — `has_action`, `cites_reference`, `explains_impact` — correspond precisely to what RAG contributes: the retrieved context provides the regulatory citation, the LLM synthesises the clinical/legal consequence, and both feed into the remediation recommendation.

The baseline explanation already scores well on `cites_rule` (6/6) and `cites_field_value` (6/6) because those are produced deterministically from the rule metadata. The gap between baseline and RAG is entirely attributable to the retrieved context.

The average RAG word count of 175 vs. 42 for baseline reflects genuinely additional information, not padding. RAG explanations consistently include: the exact field values that caused the failure, the name and section of the relevant regulation, the clinical or regulatory consequence (claim denial, fraud investigation risk, patient safety event), and a specific corrective action (verify against a named source document, correct the specific field, escalate to a named team).

7 • Challenges

7.1 LLM Resistance to Generating Invalid Records

The most persistent technical challenge was the model's resistance to generating semantically invalid records. Models trained with human feedback develop a strong prior toward logically coherent output. When prompted to generate a record with `discharge_date < admission_date`, early prompt versions produced records where the model "corrected" the dates to be consistent — the opposite of the intended behaviour.

Resolution required three prompt iterations: adding a concrete example of the violation, adding a negative constraint ("do NOT make the dates consistent"), and framing the task as intentional research data generation rather than record creation. Even the production prompt version has a ~4% non-compliance rate, requiring the semantic quality gate to regenerate rejected records.

7.2 Same-Day Discharge Edge Case

The initial implementation of HC-003 used a strict inequality (`discharge_date > admission_date`), which incorrectly flagged same-day discharges (LOS = 0 days) — a valid and common clinical pattern for outpatient procedures. This produced false-positive violations on edge-case records. The fix was to change the condition to `discharge_date >= admission_date`, with specific test cases added to cover both the same-day discharge case (valid) and the one-day-before discharge case (invalid).

This illustrates a general pattern in semantic rule development: boundary conditions between valid and invalid are where rules require the most careful specification and the most specific test cases.

7.3 Cross-Field Dependency in Confidence Scoring

The severity-weighted penalty formula does not account for interactions between violations. A record that violates both HC-001 (age mismatch) and HC-003 (discharge before admission) scores 0.40 (two critical violations, $2 \times -0.30 = -0.60$), which routes to quarantine. In practice, these two violations may share a common cause — a transposed year in the date field — and correcting one might resolve both. The current formula treats violations independently.

This is a known limitation. An interaction-aware scorer would require a more complex dependency graph and is deferred to future work.

7.4 Small Evaluation Dataset

The 16-record seed dataset is sufficient to confirm that the rules are correctly implemented but insufficient to establish statistical confidence in generalisation. With 8 records per domain (3 invalid, 5 valid), the 95% confidence interval for precision is approximately [0.29, 1.0]. The audit-log records provide a larger sample for latency and violation-frequency analysis but are not independently labeled and therefore cannot be used for precision/recall evaluation.

The full 600-record synthetic dataset (300 per domain) would address this limitation. It is designed and scaffolded; generation requires an API key and runtime of approximately 10–20 minutes.

7.5 Baseline Profiling from Small Samples

Both drift detection baselines are profiled from 3 records each. A 3-record baseline produces unreliable variance estimates for numeric fields, making z-score drift detection noisy. The standard deviation estimated from 3 observations can be zero for constant fields (no drift detectable) or arbitrarily large for high-variance fields (drift constantly reported). Production baselines should be profiled from at least 100–500 records.

8 • Ethical Considerations

8.1 Determinism and Auditability

SchemaGuard's rules are deterministic Python functions: the same input always produces the same output. Every validation decision has a full audit trail — which rules executed, which passed, which failed, the exact error messages, the confidence penalty breakdown, and the routing decision. For regulated domains, this auditability is not optional. Healthcare billing and financial lending are subject to regulatory examination; a compliance system that cannot explain its decisions in deterministic, traceable terms is not suitable for production use.

This stands in contrast to ML-based validation approaches, which can produce different outputs on identical inputs and cannot provide rule-level traces. The choice of deterministic rules over LLM classifiers is an ethical design decision: in domains where decisions affect patient care or credit access, explanatory adequacy is a requirement.

8.2 False-Positive Risk and Human Review

The false-quarantine rate of 0% on the seed dataset is encouraging but not a guarantee. A false quarantine — blocking a valid record — has downstream consequences: delayed clinical workflows, rejected loan applications, or data gaps in analytics. Any production deployment of SchemaGuard should maintain a human review process for flagged records, not rely solely on automated routing decisions.

The flagged tier (0.50–0.84 confidence) exists precisely to preserve human judgment. Records in this range are not blocked from downstream use; they are surfaced to reviewers. The confidence score and violated rules are presented alongside the record, giving reviewers the information needed to make an informed decision without starting from scratch.

8.3 Bias in Synthetic Data

The synthetic dataset generator cycles through demographic profiles but makes no systematic effort to audit for differential performance across demographic groups. If the rule engine fires at different rates on records for different age groups, genders, or income levels — because rules like HC-004 are inherently age-differential — this could produce apparent demographic disparities in quarantine rates when the system is deployed.

Before production deployment in any context where the routing decision has material consequences for individuals, a fairness audit should be conducted: validate that quarantine rates are not systematically higher for records representing protected demographic groups, controlling for rule violations.

8.4 Scope of Claims

SchemaGuard validates logical coherence, not factual accuracy. A record where the patient's diagnosis is correctly coded but is the wrong diagnosis for the underlying condition will pass all SchemaGuard rules. A loan application where all fields are internally consistent but the applicant's identity has been falsified will pass all rules. The system is not a fraud detector, a fact-checker, or a clinical accuracy validator. Using it as such would be a misapplication.

8.5 Use of Synthetic Data in Evaluation

The evaluation relies entirely on synthetic data — records generated by an LLM using prompts designed to produce specific failure modes. The rules were designed knowing what the test data would look like, which introduces circularity. An adversarial evaluation on real EHR or loan application data would provide a more honest assessment of generalisation. Access to such data for a course project is not feasible, but this limitation should be acknowledged in any production readiness assessment.

9 • Future Work

Scaled evaluation dataset. The full 600-record synthetic dataset (300 per domain, with balanced rule-specific invalid records and diverse edge cases) is designed and scaffolded. Generating it requires an API key and approximately 10–20 minutes of runtime. This would enable statistically significant precision/recall estimates with confidence intervals.

Additional domains. The rule-registry architecture is domain-agnostic: adding a new domain requires writing a JSON schema and a rules file. Candidate domains include insurance claims (temporal and coverage rules), electronic prescriptions (drug interaction and dosage rules), and structured clinical notes (SOAP format consistency rules).

Active learning loop. Records scoring in the 0.75–0.85 confidence band — near the trusted/flagged boundary — are candidates for human review and label propagation. Corrections from reviewers could be used to recalibrate rule thresholds, improving separation between valid and invalid populations over time.

LLM-assisted rule discovery. The violation patterns in audit logs can be analysed by a language model to propose new semantic rules not anticipated at design time. This "rule mining" capability would extend SchemaGuard to failure modes not yet codified, with human-in-the-loop approval before rules go live.

Real-time streaming validation. The current architecture validates records synchronously or in batches. A WebSocket endpoint would support streaming validation of records as they are generated, enabling confidence-score feedback to influence the generation process in real time.

Multi-model RAG evaluation. The RAG explanation quality was evaluated against a single model (Claude Opus). A systematic comparison across Claude, GPT-4o, and Gemini would identify whether the quality improvement is model-specific and inform model selection for production deployment.

Production-grade drift detection. Baseline profiles from 100+ records per domain would make z-score and PSI estimates reliable enough for production alerting. Scheduled nightly drift jobs against the day's batch, with webhook alerts to Slack or PagerDuty, would close the operational monitoring loop.

10 • Conclusion

SchemaGuard demonstrates that semantic validation of LLM-generated structured data is tractable, fast, and auditable. The four-stage pipeline operates below 1 ms per record, achieves perfect classification on both evaluation domains, and produces zero false quarantines. The RAG extension produces explanations that consistently cite the applicable regulation by section number and include actionable remediation steps — capabilities absent from deterministic rule-based explanations alone.

The project establishes three reusable design patterns for LLM output validation:

1. **Rule registry with severity metadata:** separating rule logic from metadata enables penalty-weighted scoring, per-rule audit trails, and domain-agnostic pipeline orchestration.
2. **Bimodal confidence scoring:** graduated scoring preserves severity information that binary pass/fail discards, enabling multi-tier routing and downstream differentiation.
3. **RAG-grounded explanations:** retrieving domain-specific regulatory context at query time produces explanations that are qualitatively superior to deterministic templates without requiring model fine-tuning.

The combination of deterministic validation, population-level drift monitoring, and LLM-augmented explanation addresses the full operational lifecycle of LLM-generated structured data: detecting errors in individual records, monitoring for systemic shifts in output distributions, and communicating failures in terms meaningful to compliance reviewers.

References

Centers for Medicare & Medicaid Services. *Medicare Claims Processing Manual, Chapter 1 §30.2: Inpatient Hospital Billing*. U.S. Department of Health and Human Services, 2024.

Centers for Medicare & Medicaid Services. *Conditions of Participation §482.24(c): Medical Record Services*. U.S. Department of Health and Human Services, 2023.

Consumer Financial Protection Bureau. *Ability-to-Repay and Qualified Mortgage Standards Under the Truth in Lending Act (Regulation Z), 12 CFR §1026.43*. 2023.

Health Level Seven International. *HL7 FHIR R4 Base Specification — Encounter Resource*. 2023.

Institute for Safe Medication Practices. *ISMP Medication Safety Alert: Annual Report*. 2023.

Joint Commission. *Comprehensive Accreditation Manual for Hospitals: Standard RC.02.01.01 — Medical Record Content*. 2024.

National Center for Health Statistics. *ICD-10-CM Official Guidelines for Coding and Reporting, FY2024*. U.S. Department of Health and Human Services, 2024.

Office of the Comptroller of the Currency. *Comptroller's Handbook: Retail Lending*. U.S. Department of the Treasury, 2023.

Peng, B., Galley, M., He, P., Cheng, H., Xie, Y., Hu, Y., ... & Gao, J. (2023). Check your facts and try again: Improving large language models with external knowledge and automated feedback. *arXiv preprint arXiv:2302.12813*.

Wei, J., Wang, X., Schuurmans, D., Bosma, M., Xia, F., Chi, E., ... & Zhou, D. (2022). Chain-of-thought prompting elicits reasoning in large language models. *Advances in Neural Information Processing Systems*, 35.

White, J. S., Fu, Q., Hays, S., Sandborn, M., Olea, C., Gilbert, H., ... & Schmidt, D. C. (2023). A prompt pattern catalog to enhance prompt engineering with ChatGPT. *arXiv preprint arXiv:2302.11382*.